

Assignment 3 Bridge Pattern

Arnur Kozhabergen, se-2529

Topic A: Drawing

GitHub: <https://github.com/arnur-kozhabergen/sdp.asmnt3>

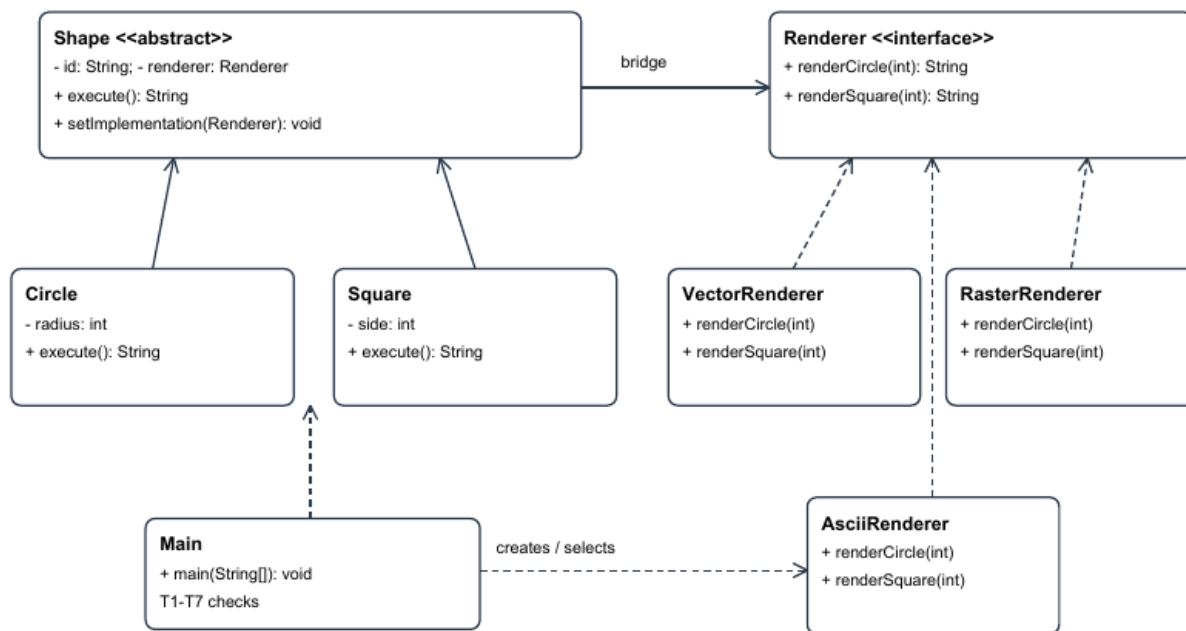
Base commit: f68abb985a57a5e0be449c3846f21904c680f6a8

Source commit: 70cd86e37c29d406fc86c9748249767409cb420b

1 Introduction

I made a small drawing program with Circle and Square. Each shape keeps its own size. A Renderer decides how the result looks. I used Bridge because the same shape can work with vector, raster or ASCII rendering without a class for every combination.

2 UML diagram



Shape stores a Renderer reference. Circle and Square extend Shape. The three renderers implement Renderer. Main creates the objects and runs T1 to T7.

3 Clean Code examples

1 Separation of responsibilities. Renderer defines the low-level operations. Circle and Square provide the shape data.

```
public interface Renderer {
    String renderCircle(int radius);
    String renderSquare(int side);
}
```

2 Meaningful names. The class and method tell me which renderer and shape are used.

```
public class VectorRenderer implements Renderer {
    @Override
    public String renderCircle(int radius) {
        return "VECTOR circle radius=" + radius;
    }
}
```

3 Small methods. Circle.execute only passes its radius to the renderer.

```

public String execute() {
    return renderer().renderCircle(radius);
}

```

4 No duplicated checking logic. Main.check compares the actual and expected results for six checks.

```

String actual = shape.execute();
boolean success = expected.equals(actual);
if (success) {
    passed++;
}

```

5 Encapsulation. The ID stays fixed, and the renderer changes only through setImplementation.

```

private final String id;
private Renderer renderer;
public void setImplementation(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer, "renderer");
}

```

4 Results

```

T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3
T5 PASS | Circle + VectorRenderer -> RasterRenderer | sameObject=true | stateUnchanged=true
    before=VECTOR circle radius=2 | after=RASTER circle radius=2
T6 PASS | Circle + AsciiRenderer | result=ASCII (circle radius=2)
T7 PASS | Square + AsciiRenderer | result=ASCII [square side=3]
SUMMARY: 7/7 PASS

```

5 Extension and comparison

The base commit passed T1 to T5 with VectorRenderer and RasterRenderer. Then I added AsciiRenderer and updated Main for T6 and T7. The extension.diff file shows that the old Bridge classes were not changed. Bridge separates two dimensions during design. Adapter is used when existing interfaces do not match. The trade-off here is more small classes and one extra call through Renderer.

6 Conclusion and references

All seven checks passed. T5 uses == to confirm the same Circle object and checks that its ID and radius did not change after switching the renderer.

References: Assignment 3 Bridge Pattern instructions; Refactoring.Guru Bridge and Adapter pages, accessed 4 October 2026.